Growth Portal Course Proposal

Observability — 3 Pillars, from Structured Logging to Failure Detection

Ad-hoc Activity Plan, Item 2 · Prepared by Leander · 2026-08-24, revised 2026-08-26 after EM feedback

Objective

Create and post a course to the Growth Portal on **observability** — the Growth Area identified in my 2-week training plan. The course translates what the training plan covered (structured logging, metrics, distributed tracing, and silent-failure detection) into a short, practical course for other Full Scale engineers, with an accessible entry point for people outside engineering (QA, UI/UX, PM) who work alongside these systems.

Approach

The course teaches **observability concepts**, demonstrated in a real NestJS application built during the training plan — the framework is the vehicle for showing the ideas concretely, not the subject of the course itself. Every technical module explains the underlying mechanism in plain language before any code appears on screen, so the reasoning transfers even to someone working in a different stack.

Because the audience is mixed, the course is structured in two tiers: one short, jargon-free module anyone can follow, followed by technical modules that assume an engineering background.

Response to feedback

Six points on the previous draft — addressed below, and reflected in the module outline and schedule that follow.

Feedback	Response
Minimize jargon	Every concept is explained in plain language first; the field term is named only once that explanation has already landed — never the reverse.
Avoid framing the content around "gaps"	Reframed every module as "here's the team's standard," not "here's what you don't know" — the same content, presented as a shared convention rather than a deficiency.
Research the landscape; name specific topics	Considered and ruled out: alerting/on-call, SLOs/error budgets, log aggregation platforms, APM/RUM, synthetic monitoring, chaos engineering, cardinality as a standalone topic. The 4 modules below are what's kept, plus a host-metrics addition to Module 3.

Explain when each approach should and shouldn't be used	Should/shouldn't framing added to all 5 technical modules, including the new host-metrics segment (compute-bound vs. I/O-bound — host metrics answer the first question, not the second).
Demonstrate structured logging across source, config, external services, and infrastructure	Audited the existing demo app against all four categories. Source code and external-service logging were already solid; application-configuration and infrastructure logging had no coverage, since the demo app has no database or config validation. Rather than build a second project, the existing one is being extended: a Postgres instance with a connection/health-check log, and env-schema validation with a log of what got validated.
Recording estimate too optimistic	Added a build day (Day 0, below), split the combined Modules-2-and-3 scripting day into two, and split the recording day itself — Module 3 alone, then the other four — so no single day cuts into the time reserved for other ad-hoc work against a fixed 8-hour workday, and so a retake spiral on the riskiest module doesn't threaten the rest.

Module outline

Tier	Module	Covers
Everyone	1. Why observability, and the 3 pillars	No jargon, no code — what a system looks like with no observability vs. with it, and why that gap matters.
Engineers	2. Structured logging + correlation IDs	Tracking one request's behavior across a service without losing context mid-flight — and applying the same structured format outside request handlers, to config validation and infrastructure connection lifecycle.
Engineers	3. Metrics that survive scale	Designing metrics that stay cheap to store and query as traffic grows. Plus: when host-level metrics (CPU/RAM) help diagnose a slow service, and when they can't — the difference between a service that's compute-bound and one that's waiting on something else.
Engineers	4. Distributed tracing	Seeing how one request moves across multiple services, not just within one.
Engineers	5. Detecting silent failure	Catching the failures that produce no error at all — the hardest failure mode to design for.

Format

A recorded, produced video course (scripted, edited, not a single continuous take) posted to the Growth Portal, with each module labelled by tier so viewers can self-select how deep to go.

Execution schedule

Revised to 9 days against a fixed 8-hour workday, with each day's course-hours sized to leave real headroom (targeting ~2h) for the other ad-hoc items (SkillIQ, automation project proposal) the same day, rather than treating them as scheduled separately. Day 0 is new — the extension work from the feedback response above. The old combined Modules-2-and-3 scripting day is split into two (Days 2 and 3), and the recording day is split into two as well (Day 5: Module 3 alone; Day 6: the other four) — Module 3 carries the highest retake risk (live diagram narration, host-metrics demo), and recording is specifically where perfectionism tends to eat time, so it gets isolated room rather than sharing a fixed budget with the rest.

Day	Focus	Output
0	Extend the demo app — Postgres + health-check log, env-schema validation, host-metrics exporter + dashboard panel (implementation delegated; this day is review and verification)	Demo app covers all 4 structured-logging categories from the feedback; stack verified working
1	Outline + Module 1 scripting	Course outline finalized; Module 1 script drafted and reviewed
2	Module 2 scripting only	Script drafted, read aloud and checked for framework-tutorial drift
3	Module 3 scripting only, including the host-metrics addition	Script drafted, read aloud and checked
4	Modules 4–5 scripting, plus review of the host-metrics wiring diagram used in Module 3	Scripts drafted and checked; full run-through of all 5 for pacing
5	Recording — Module 3 alone	Module 3 recorded, re-takes allowed without pressure from the other four
6	Recording — Modules 1, 2, 4, 5	Remaining four modules recorded, re-takes as needed
7	Editing + self-review	Cut, tier labels added, reviewed against each module's objective
8	Review + publish	Reviewed with EM, feedback incorporated, published to Growth Portal

Items 3 (SkillIQ) and 4 (automation project proposal) from the same ad-hoc plan are self-paced and run within the daily headroom above, not stacked on top of this schedule as additional days.
